

Permit conversions to arrays of unknown bound

ISO/IEC JTC1 SC22 WG21, Evolution Working Group

P0388R0

Permit conversions of arrays of known bound to pointers or references to arrays of unknown bound. This is analogous to evolution issue 118.^{[EWG118](#)}

Robert Haberlach (rh633{at}cam{dot}ac.uk)

2016-06-28

Motivation and impact

As of core issue 393,^{[CWG393](#)} function parameters can be pointers or references to arrays of unknown bound. However, binding such a parameter to an array of known bound isn't permitted:

```
void f(int(&)[]);
int arr[1];

f(arr);           // Error
int(&r)[] = arr;  // Error
```

This restriction is unjustified and should be removed. The practical impact of this extension is deemed insignificant; such parameters were only allowed a while ago (with GCC not yet supporting them), and SFINAE-dependent code that is negatively influenced by this change is most certainly not existent.

Approach and decisions

The initialization of pointers to arrays of unknown bound will be allowed by introducing a corresponding pointer conversion. Reference initialization rules will be adjusted by modifying reference compability. Such bindings have Exact Match rank.

We also propose to allow list-initialization for references to arrays of unknown bound by deducing the array temporary's size.

Ranking of reference initialization conversions

Consider

```
void f(int(&)[ ]),      // (1)
     f(int(&)[1]),      // (2)
     f(int*);           // (3)

void h(int(*)[ ]),      // (a)
     h(int(*)[1]);      // (b)
```

(2) and (b) should clearly be better than (1) and (a), respectively, as the former overloads are more restricted.

Furthermore, (3) should be equal to (1) (as it is to (2)). To maintain this ambiguity, (1) must be given Exact Match rank. We do not favor this ordering and feel that (3) is conceptually less specialized than (2) and even (1); however, consistency is more important than fixing part of the problem for arrays of unknown bounds.

Finally, (a) should be worse than (b), which is implied by (a) necessitating a pointer conversion in that case.

Ranking of list-initialization sequences

We also propose to allow list-initialization and introduce corresponding rules in overload resolution:

```
int b(int (&&)[ ] );    // #1
int b(long (&&)[ ] );    // #2

int b(int (&&)[1]);      // #3
int b(long (&&)[1]);     // #4

int b(int (&&)[2]);      // #5

b({1});
```

Here,

- #1 and #3 should rank better than both #2 and #4, as no promotion is necessitated.
- #1 should rank worse than #3, being far less specialized.
- #1 and #2 should rank better than #5, as the latter requires a larger array temporary.
(#3 and #4 also rank better than #5 for the same reason, which has been dealt with by core issue 1307,^{[CWG1307](#)} whose resolution this paper extends).

For these reasons, the size of the referenced array is the primary criterion, followed by the worst performed conversion and finally the

known/unknown bound tie-breaker.

Proposed wording

☐ Hide deleted wording

Create 4.10 [conv.ptr] ¶4 with the following content:

A prvalue of type “pointer to array of N cv T” can be converted to a prvalue of type “pointer to array of unknown bound of cv T”. The result is a pointer to the array. The null pointer value is converted to the null pointer value of the destination type.

Modify 8.5.3 [dcl.init.ref] ¶4 as follows:

Given types “cv1 T1” and “cv2 T2”, “cv1 T1” is reference-related to “cv2 T2” if T1 is the same type as T2, ~~or~~ T1 is a base class of T2 **or T1 is an array type of unknown bound of U and T2 is an array type of known bound of U.**

Modify 8.5.4 [dcl.init.list] ¶(3.8) as follows:

Otherwise, if T is a reference type, a prvalue temporary ~~of the type referenced by T~~ is copy-list-initialized or direct-list-initialized, depending on the kind of initialization for the reference, and the reference is bound to that temporary. **The type of the temporary is either the type referenced by T or, if T is “array of unknown bound of U”, the type “array of N U”, where N is the number of elements in the initializer list.**

Modify 13.3.3.1.5 [over.ics.list] ¶5 as follows:

Otherwise, if the parameter type is “array of N X” **or “array of unknown bound of X”**, if there exists an implicit conversion sequence ~~for each element of the array from the corresponding~~ **from each** element of the initializer list ~~(or and from { } if there is no such element~~ **N exceeds the number of elements in the initializer list)** **to X**, the implicit conversion sequence is the worst such implicit conversion sequence.

Augment 13.3.3.2 [over.ics.rank] ¶(3.1) as indicated:

List-initialization sequence L1 is a better conversion sequence than list-initialization sequence L2 if

- L1 converts to `std::initializer_list<X>` for some X and L2 does not, or, if not that,
- L1 converts to type “array of N1 T”, L2 converts to type “array of N2 T”, and N1 is smaller than N2, **or if not that, L1 converts to an array of unknown bound and L2 converts to type “array of N X”, where N is greater than the number of elements in the initializer list,**

even if one of the other rules in this paragraph would otherwise apply. [Example:...— end example]

Create 13.3.3.2 [over.ics.rank] ¶5 with the following content:

If none of the previous rules in this section apply, two list-initialization sequences are indistinguishable except that a list-initialization sequence that converts to an array of known bound is better than one that converts to an array of unknown bound. [Example:

```
void f(int    (&&)[ ] ),    // #1
    f(double (&&)[ ] ),    // #2
    f(int    (&&)[2]);     // #3
```

```
f( {1} );           // Calls #1
f( {1.0} );         // Calls #2
f( {1.0, 2.0} );    // Calls #2: Identity conversion is better than floating-integral conversion
f( {1, 2} );        // Calls #3: Converting to array of known bound is better
```

— end example]

Acknowledgments

The author would like to thank David Krauss for guidance and support.

References

[EWG118] “[tiny] Allow conversion from pointer to array of known bound to pointer to array of unknown bound”: wg21.link/ewg118

[CWG393] “Pointer to array of unknown bound in template argument list in parameter”: wg21.link/cwg393

